

miniagent 完整报告

最小完备 Coding Agent 的设计、实现与验证。对应仓库 github.com/aitachi/miniagent(commit 690eabf,2026-08-17)。核心约 1,500 行 (15 个 Python 文件、13 个实质模块),唯一第三方依赖 requests;Python 3.10+,Docker 可跑,默认后端 qwen3.7-max(30 万上下文)。本报告对齐 LeetCode 合并严格版版式(黑白灰,无彩色渲染);每个模块配知识图谱矢量流程图 + 分层工作流表 + 分层描述;另设跨模块交互专节(token 漂移校准 · 多源会话 · context × memory)。

对齐最新代码:本版含 token 漂移校准(DriftCalibrator 锚点反馈)与 resume 协议对修复;65/65 离线回归;并反映 690eabf「isolate sessions and align module contracts」—— ToolRuntime/HookManager 实例化(会话不再串绑);compact/overflow 事件携带消息快照(resume 不再回退到压缩前);会话文件名独占创建防并发碰撞;sandbox 只读名单与注册表同源;web 并发闸超时后保持占用。 历史遗留:2026-08-16 压测发现的「跨进程 resume 丢失压缩摘要」已在本版修复。

目录(7 大板块 / 13 模块)

- 1. 概览(设计宪法 · 三层架构 · 验证证据)
- 2. 模块设计(13 模块:分级表 + 每模块流程图 + 分层工作流表 + 分层描述)
- 3. 如何运行(环境变量 · 三形态 + Web · 扩展点)
- 4. 对比报告(miniagent vs kimi-code vs dsh vs Hermes)
- 5. 问题与解决(实测问题清单 · 已知边界)
- 6. 工程问答(16 题,三面)
- 7. 压力测试与问题发现(2026-08-16 实测 + 修复对照)

1、概览

miniagent 是一个能自动写代码的 Agent Harness:把一个只会生成 token 的模型,变成一个能读文件、跑命令、记教训、守边界的工程实体。两条设计宪法贯穿所有模块。

宪法一 · Prompt Cache 神圣 —— 同一 Agent 会话内 system 前缀冻结;动态内容(记忆、steer)一律注入消息尾部;正常步骤只追加,压缩则以显式快照事件替换中段。

宪法二 · 窄腰核心 —— 核心 = 一个循环 + 一个消息列表 + 一个工具调用协议;沙盒/记忆/技能/Hooks/ACP/Web 全部挂在边上。

1.1 三层架构

表面层	cli.py(终端交互/一次性任务) · web.py(HTTP 查询表单) · acp.py(stdio JSON-RPC 宿主)
Harness	loop.py(窄腰核心 Agent:while ask→act→append) - context.py 每步 preflight 压缩(85% 阈值/头尾保留/免疫包裹) - tools.py Agent 实例级 ToolRuntime + 11 工具 + 校验 + 50K 落盘治理 - sandbox.py 注册表同源工具集→hardline→三分级→路径工具围栏 - hooks.py Agent 实例级 HookManager + 进程外 hook(30s 强杀)+ stop 门锁 - memory.py MEMORY.md 冻结快照 + provenance 追加 + 注入安检 - skills.py SKILL.md 索引常驻一行 + 正文按需注入 - subagent.py delegate_task 隔离派生(独立上下文/预算/transcript) - session.py JSONL transcript + 压缩快照重放(坏行容错)
模型层	llm.py → qwen3.7-max(OpenAI 兼容,429/5xx 指数退避重试)

1.2 验证证据(全部现场实测)

验证	结果
本次离线模块回归(FakeLLM/Mock,覆盖 13 个模块 + 漂移校准 + 协议对修复)	65/65 OK
本次在线模型回归	未运行:dev-server 未配置 MINIAGENT_API_KEY,未借用其他项目密钥
现场写项目 taskcli(待办 CLI)	18 tests OK,agent 自愈空 JSON bug 并沉淀教训
Docker 内现场写项目 expense-cli	15 tests OK,干净容器复跑一致
5 个复杂压力任务(T1–T5)	39/19/54/26 tests OK,73 次工具调用 0 错误
hook 阻断适配探针	pip install 被进程外 hook 阻断→模型不重试、改标准库完成
大文件结果治理	241KB 日志→模型只见 2,130 字符 preview,全量落盘
--resume 会话恢复	重放后模型准确回忆上轮文件名与统计结果

上下文压缩实战(小窗口强制)	压缩真实触发 2 次,免疫包裹注入,任务完成
delegate_task 子代理	父子 transcript 分离;子代理 11 次探索仅回传蒸馏报告
密钥卫生	子进程环境剥离 KEY/TOKEN 类变量;工作区零密钥痕迹

2、模块设计

每个模块 = 一个文件 = 教程一章的「最小设计」+ 必要的第 1~2 级演化;每份复杂性都对应一个真实痛点。分级:[L0] 骨架(能跑) · [L1] 健壮(失败路径处理) · [L2] 生产化(生产事故反推的机制) · [L3] 演化(对照 kimi/Hermes 的下一级,按需取不强上)。

2.1 模块分级表(13 模块)

#	模块	行	职责一句话	分级	防什么事	参照物
1	loop.py	231	主循环 ask→act→append,唯一窄腰核心	[L2]	死循环烧钱、崩溃丢进度、不可回放	kimi 主循环
2	context.py	118	每步 preflight 压缩(85% 阈值)	[L2]	token 溢出即死;粗暴截断打碎缓存	kimi 阈值+头尾保留
3	tools.py	372	实例级 runtime+11 工具注册/校验/治理	[L2]	会话串绑、schema 漂移、超长结果灌窗	kimi 截断+落盘
4	sandbox.py	120	注册表同源名单+hardline+权限+路径围栏	[L3]	未知工具、危险命令、路径越界	Hermes hardline 先于 yolo
5	memory.py	103	冻结快照+追加+快照/搜索注入安检	[L2]	跨项目串记忆、毒记忆劫持新会话	kimi AppendLogStore
6	skills.py	86	索引一行常驻,正文按需取	[L1]	提示词膨胀撑爆前缀缓存	kimi 三层披露
7	hooks.py	166	实例级 5 事件+进程外 hook+门锁	[L2]	会话串 hook、hook 崩溃、stop 死锁	kimi stopHook 门锁
8	acp.py	104	stdio JSON-RPC 多 session 分发	[L1]	一条 print 毁掉协议流、session 串线	ACP 协议
9	session.py	93	JSONL 消息/压缩快照重放+TTL	[L2]	压缩后 resume 回退;并发文件名碰撞	kimi append-only
10	subagent.py	38	delegate_task 上下文隔离派生	[L1]	探索污染主线;父子压缩分叉	kimi 扁平子代理
11	llm.py	114	OpenAI 兼容+退避重试+溢出自愈	[L2]	429/5xx、arguments 解析失败	kimi 溢出自愈≤3
12	cli.py	106	交互/一次性/ask 三形态	[L1]	GBK 乱码、无恢复入口	—
13	web.py	171	HTTP 表面+独立 workdir+并发闸	[L1]	请求串目录、超时后并发闸失效	—

2.2 整体 workflow(知识图谱)

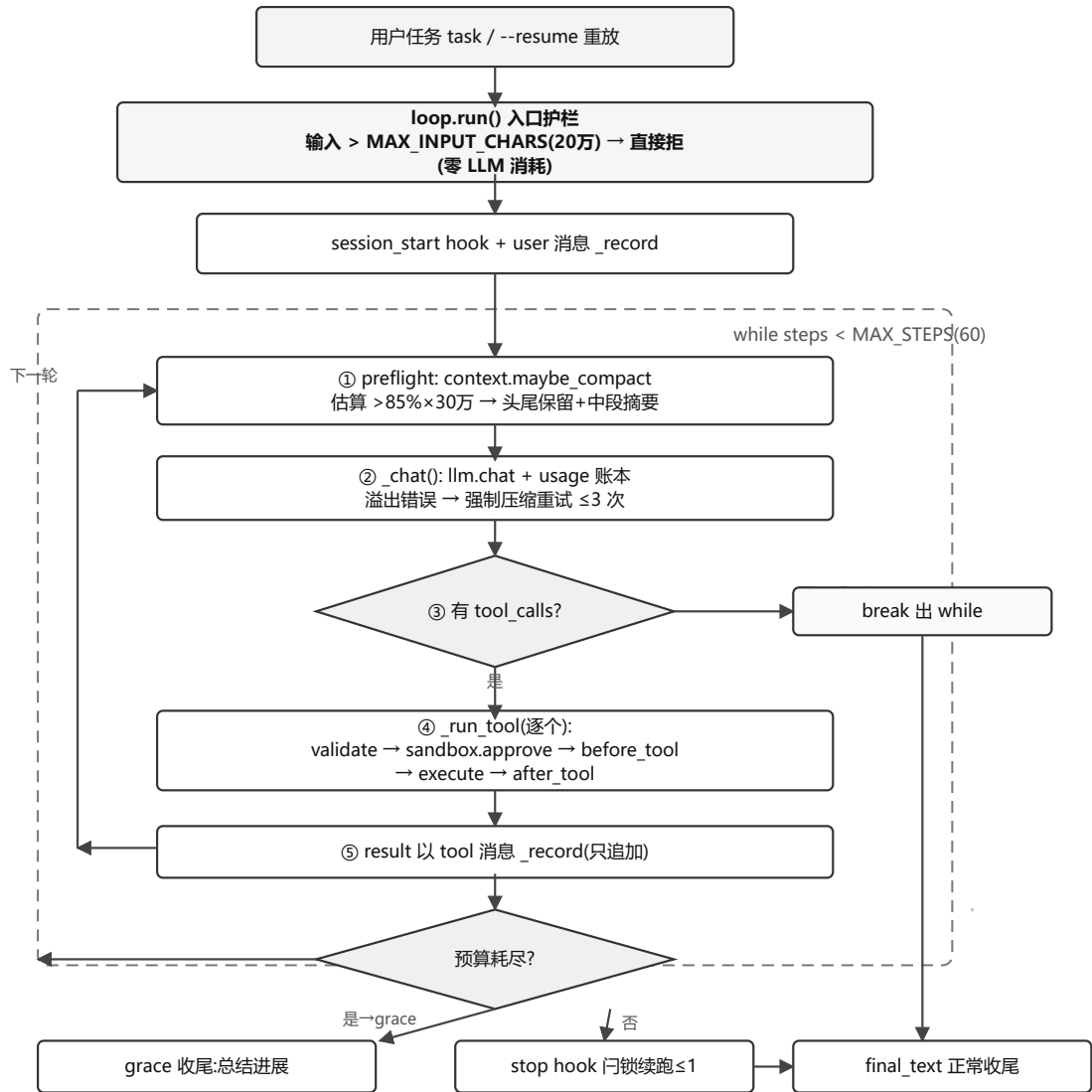


图 1 loop.py 窄腰主循环:入口护栏 → preflight → chat → tool_calls 裁决链执行 → append 回环;预算耗尽 grace 收尾;stop 门锁续跑 ≤1 次

为什么这样设计:主线是中间纵列的一次 turn。每一步先过 context.py 的 preflight(窗口没超就原样放行,前缀不动即缓存命中),再把消息列表发给模型;模型要么回文本(不调工具→循环结束),要么回 tool_calls。每次工具调用依次经过注册/参数校验 → hardline/权限裁决 → before_tool hook;未知、畸形或危险调用不会进入 before_tool/handler,after_tool 仍统一收到失败结果。任何一环说「不」都把错误文本回填给模型自行换路 —— 这是 miniagent 最核心的自愈语义。

一个真实 turn 的走读(历史压测任务,30 万窗口被真实逼近):模型先 read_file 一个 240KB 日志 → 结果治理触发,全量 255,935 字符落盘,模型只见 2,130 字符 preview + 文件路径 → 模型按 preview 判断再 grep 定位 → 若干步后 preflight 估算超 85%,中段被摘要压缩(免疫包裹注入,system 与首条 user 原样保留)→ 任务继续直到最终总结。消息与压缩后的消息快照都写入 transcript;--resume 按事件顺序恢复压缩后的真实上下文,而不是回到压缩前。

2.3 逐模块详述(每模块:知识图谱流程图 + 分层工作流表 + 分层描述)

模块 1 • loop.py 主循环(231 行) [L2] • 层:主管

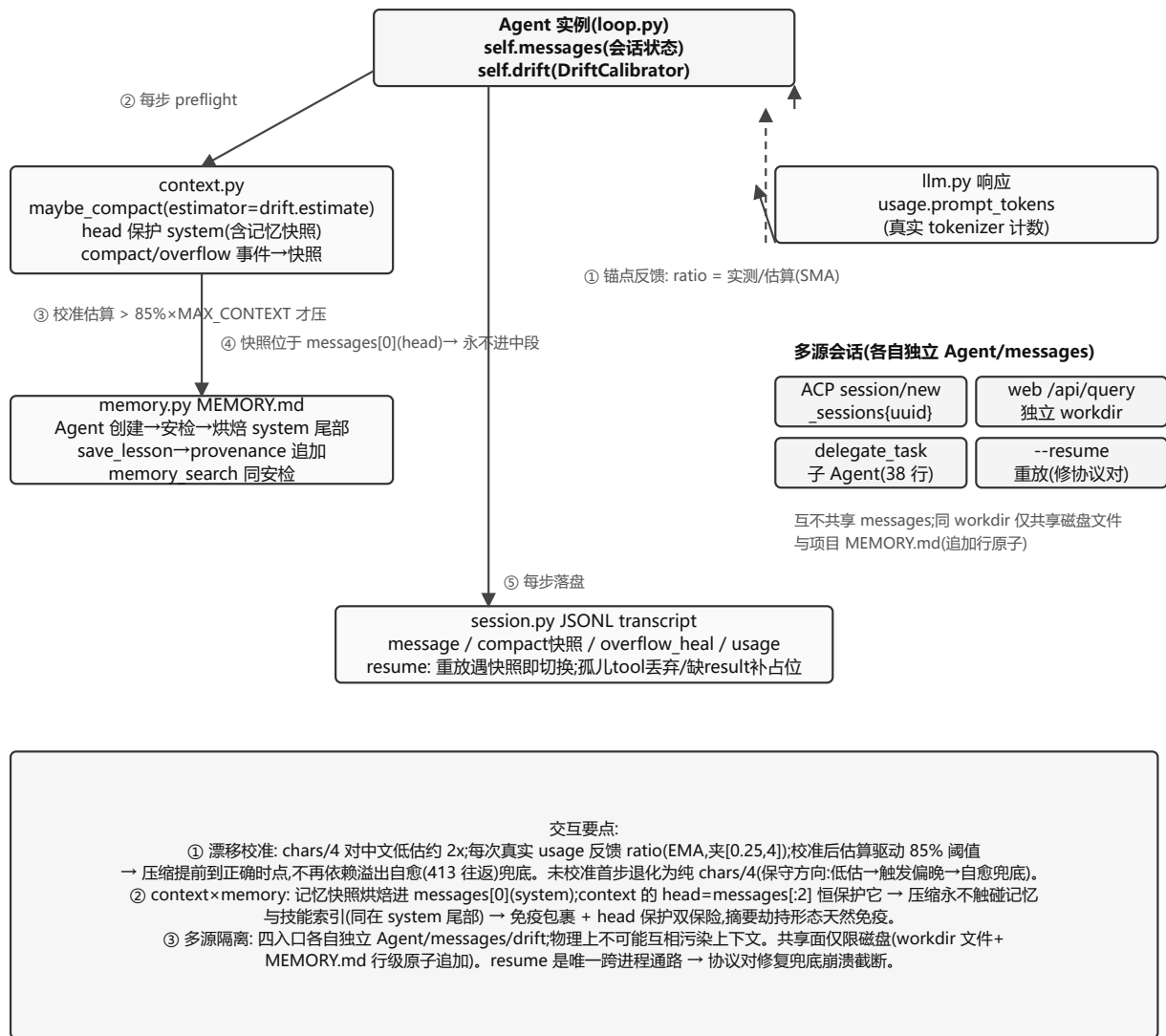


图 1-1 loop.py 与 context/memory 的交互视图(与图 1 主流程图互补:图 1 看控制流,本图看数据流——估算/快照/记忆三向交互)

层	输入	输出	处理(关键步骤)
①入口	user_text / --resume transcript	拒绝文案或进入主循环	len > MAX_INPUT_CHARS(20万) 直接拒,零 LLM 消耗;resume 则 reconstruct_messages 重建(含压缩快照)
②会话开启	workdir + task	session_start 事件 + user 消息	hooks.fire(session_start);_record(user)逐条落 transcript
③主循环	messages(上轮)	assistant / tool 消息	每步先 _compact() preflight → _chat() → 无 tool_calls 即 break; 有则逐个 _run_tool → result_record → 回环
④预算	steps 计数	grace 收尾文本	steps ≥ MAX_STEPS(60) → 追加一条「请总结」的 user → 最后一次 chat,不再执行工具
⑤终止	final_text	stop hook / session_end	stop 阻断且门锁可用 → 注入 reason 续跑(每 turn 仅一次);否则 fire(session_end) + usage 落盘

分层描述:入口层拦截恶意长文本;会话层发事件并记录首条消息;循环层承担 ask→act→append 全部语义;预算层兜底失控;终止层保证 stop hook 不会把 turn 变成永动机。四条不变量:消息列表是会话状态;终止由模型决定;工具错误作为文本回填让模型自愈;正常步骤只追加、压缩以显式快照替换中段。

为什么这样设计:把「循环」本身压缩到能穷举测试的体量,所有策略(压缩/安全/记忆)都以 preflight 或事件的形式插在循环的固定点上——换 provider、换存储、换表面都不需要动 loop.py,这是「窄腰」的具体含义。

模块 2 • context.py 上下文压缩(118 行) [L2] • 层:治理

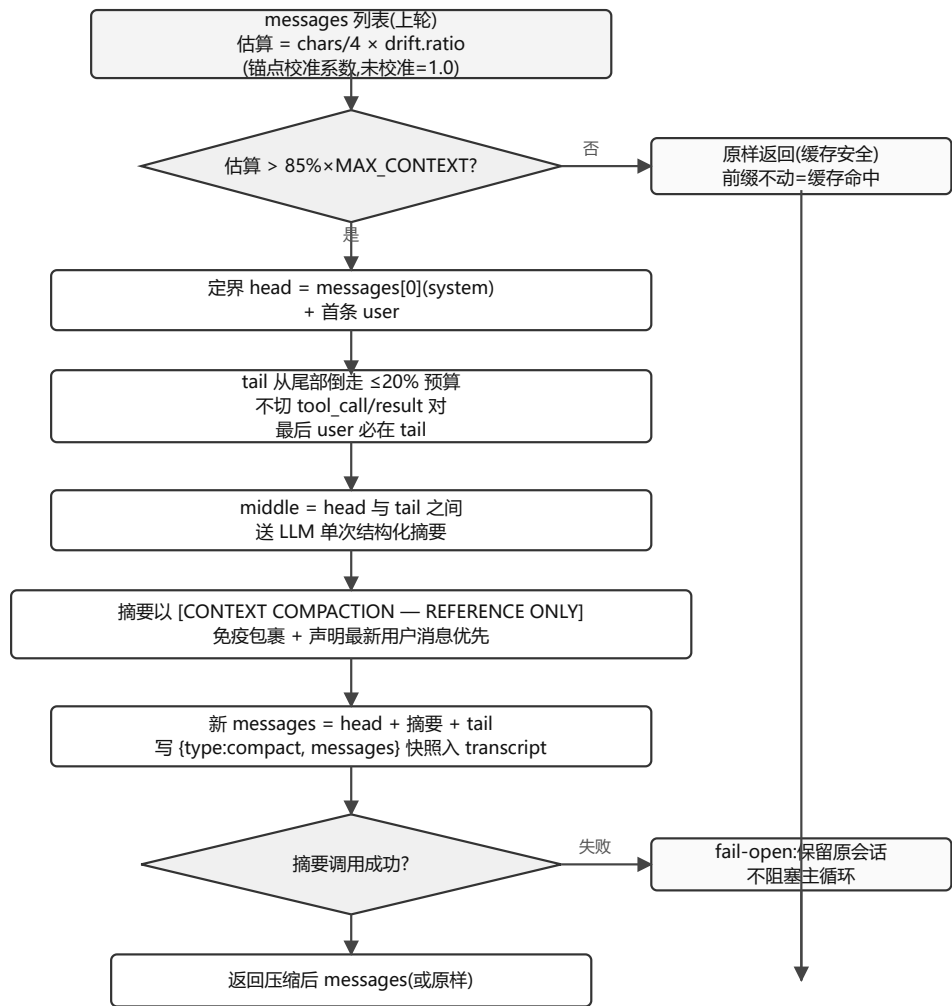


图 2-1 context.py 知识图谱:估算 → 85% 阈值 → head/tail 定界 → 中段摘要 → 免疫包裹 → 快照替换

层	输入	输出	处理(关键步骤)
①估算	messages + MAX_CONTEXT + drift.ratio	校准后估算 tokens	chars/4 × 漂移系数;系数由每次真实 usage.prompt_tokens 锚点反馈(EMA,夹[0.25,4]);未超 85% 原样返回 —— 前缀不动,缓存命中
①'漂移校准	provider usage.prompt_tokens	drift.ratio 更新	每次 _chat 后:ratio = 实测/估算,指数滑动平均 α=0.3,单点夹 [0.25,4];中文场景 chars/4 低估约 2x,不校准则压缩触发偏晚,靠溢出自愈兜底(413 往返)
②定界	超阈值 messages	head / middle / tail 三段	head = system(含记忆快照+技能索引)+ 首条 user;tail 从尾部倒走 ≤20% 预算,不切 tool_call/result 对,最后一条 user 必在 tail
③摘要	middle 文本	单段结构化摘要	LLM 单次摘要(摘要调用本身也产生 usage 锚点);失败 fail-open 保留原会话,不阻塞主循环
④免疫包裹	摘要文本	带包裹的摘要消息	[CONTEXT COMPACTION — REFERENCE ONLY] + 声明最新用户消息优先,防摘要旧指令被当成现行指令
⑤替换	head+摘要+tail	新 messages(写回调用方)	调用方(loop)把新列表连同 {type:compact, messages} 快照写入 transcript —— resume 恢复压缩后真实上下文

分层描述:校准层回答「估得准不准」(真实 usage 锚点持续纠偏 chars/4 的中文低估);估算层回答「要不要压」;定界层回答「保什么、压什么」—— head 恒含记忆快照与技能索引,压缩永不触碰;摘要层回答「怎么压」;包裹层防劫持;替换层保证 resume 能恢复压缩后的真实上下文。

为什么这样设计:不切协议对是因为 OpenAI 协议里 tool_call 与其 result 必须成对出现,从中间切断直接 400 报错;85% 阈值留出的 15% 余量,正是给单条大工具结果(最坏 2K preview + 若干条消息)的缓冲。**token 漂移:**chars/4 对中文低估约 2x,本版加 DriftCalibrator —— 每次真实 usage 锚点反馈(EMA α=0.3,单点夹 [0.25,4]),校准系数直接驱动 85% 阈值判定与 tail 预算。这是 kimi 「锚点实测+估算」双层计量的最小版:不引入阻塞式测量 job,用顺路得到的 usage 做零成本校准。

模块 3 • tools.py 工具运行时(372 行) [L2] • 层:执行

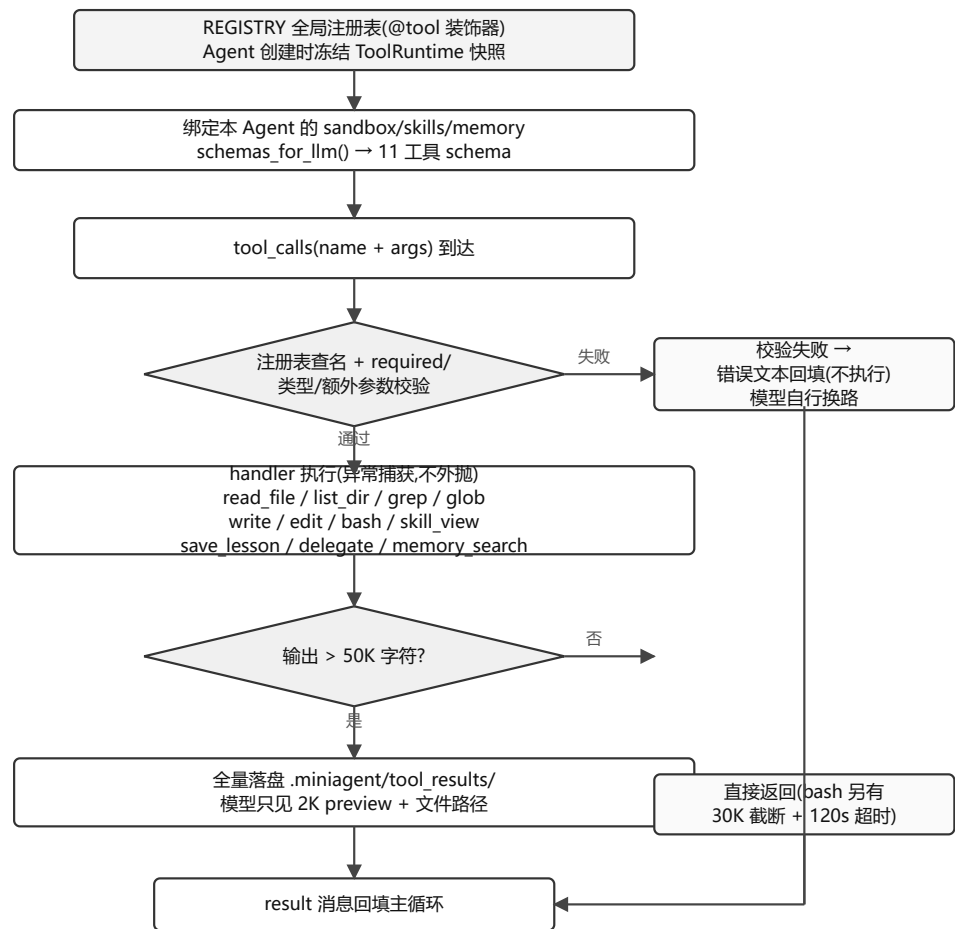


图 3-1 tools.py 知识图谱:全局注册表 → Agent 冻结 ToolRuntime → 校验 → 执行 → 结果治理(50K 落盘/2K preview)

层	输入	输出	处理(关键步骤)
①注册	@tool(name, desc, schema, readonly)	REGISTRY 条目	注册必须在 Agent 创建前;重名/非法名/非 object schema 直接报错(fail-fast)
②快照	Agent.__init__	ToolRuntime 实例	冻结 schema/handler/readonly 快照 + 绑定本 Agent 的 sandbox/skills/memory;运行中新增工具只对之后创建的 Agent 生效
③校验	tool_calls(name+args)	通过或错误文本	required/类型/额外参数逐项校验;校验失败不执行;错误文本回填(不触发 before_tool)
④执行	合法调用	result 消息	handler 执行,异常捕获不外抛;顺序 = sandbox → before_tool → handler → after_tool
⑤治理	原始输出	2K preview(+落盘路径)	输出 >50K 字符全量落盘 .miniagent/tool_results;/bash 另有 30K 截断 + 120s 超时

分层描述:注册层维护唯一事实源;快照层把「这一会话能用哪些工具」固化,防会话间串绑与 schema 漂移;校验层挡畸形调用;执行层保证异常不炸循环;治理层保证单条输出灌不爆 30 万窗口。11 个内置工具:read_file / list_dir / grep / glob / write_file / edit_file / bash / skill_view / save_lesson / delegate_task / memory_search。

为什么这样设计:结果治理的 2K preview 是「模型按需再取」协议:模型看到 preview 判断是否 grep 定位,而不是把 24 万字符直接灌进窗口 —— 实测 T5 单条 read_file 结果 25 万字符被治理在 2,130 字符。

模块 4 • sandbox.py 沙盒权限(120 行) [L3] 最小版 • 层:裁决

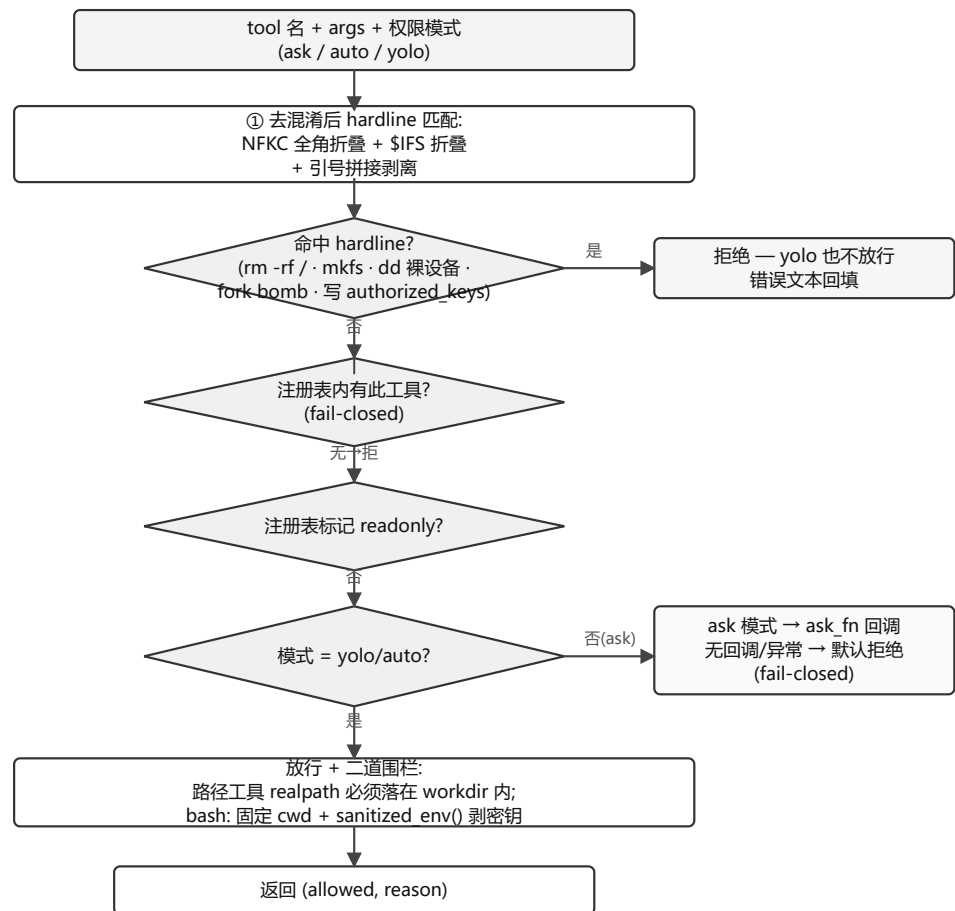


图 4-1 sandbox.py 知识图谱:去混淆 → hardline → fail-closed → 只读/模式分级 → workdir 围栏 + 环境剥离

层	输入	输出	处理(关键步骤)
①去混淆	tool 名 + args	规范化后的匹配串	NFKC 全角折叠 + \$IFS/\${IFS} 折叠 + 引号拼接剥离,再做 hardline 匹配
②hardline	匹配串	拒绝或放行	rm -rf / · mkfs · dd 裸设备 · fork bomb · 写 authorized_keys 等,yolo 也不放行
③fail-closed	工具名	未知即拒	注册表快照内无名 → 拒;调用方也不能用参数把写工具伪装成只读
④分级	readonly 标记 + 模式	允许/询问/拒绝	readonly 默认放行;yolo/auto 放行;ask 走回调,无回调/异常默认拒绝
⑤围栏	路径参数 / bash	受控执行	路径工具 realpath 必须落在 workdir 内(防符号链接逃逸);bash 固定 cwd + sanitized_env() 剥 KEY/TOKEN/PASSWORD/SECRET/CREDENTIAL

分层描述:去混淆层保证规则匹配的是命令的真实语义;hardline 层是不可让渡的底线;fail-closed 层保证白名单外无路可走;分级层把「用户授权」变成显式模式;围栏层是最后一道物理边界。注意边界:bash 只是规则裁决 + 固定 cwd + 环境剥离,不是 OS 级文件系统隔离;无人值守应套容器/E2B。

为什么这样设计:「用户已授权」不能成为无恢复路径破坏的通行证 —— 用户可能被钓鱼、被注入、或只是没意识到命令后果。危险检测是工程层的不可让渡职责,所以 hardline 在裁决链第一环,先于任何授权模式。

模块 5 • memory.py 长期记忆(103 行) [L2] • 层:会话边界

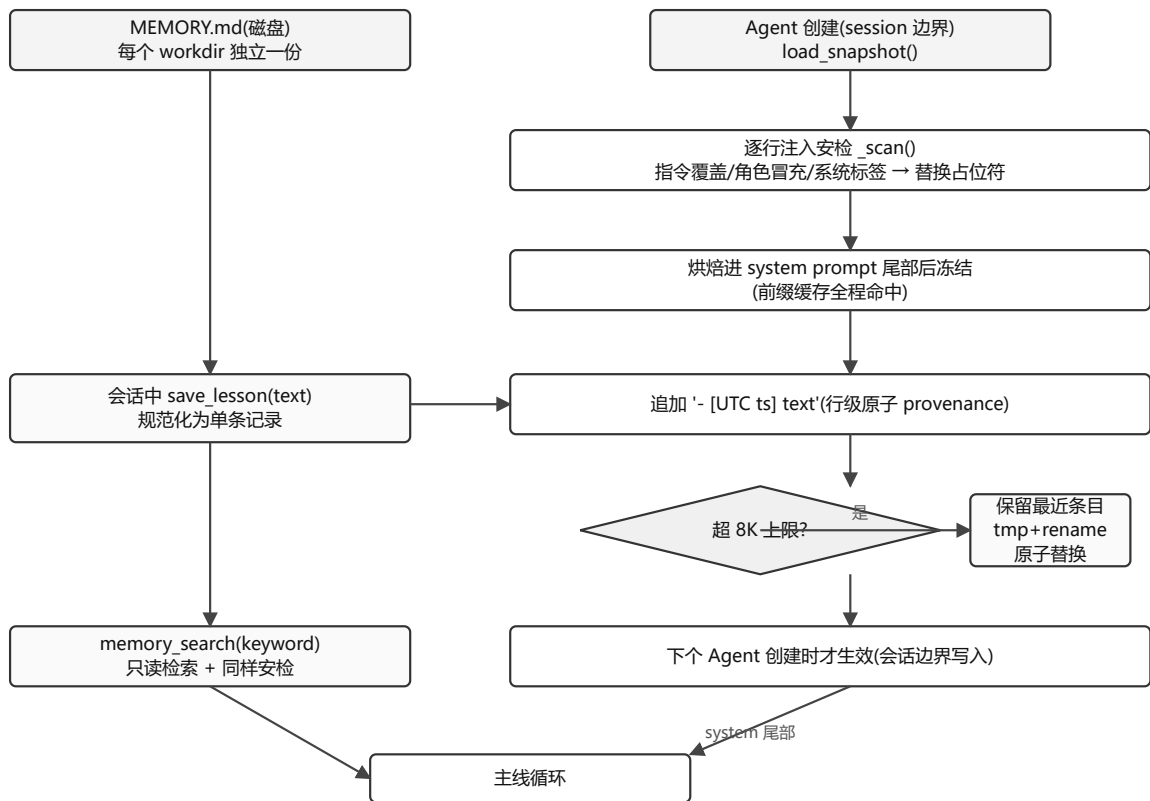


图 5-1 memory.py 知识图谱:load_snapshot 安检 → 烘焙 system 尾部冻结;save_lesson provenance 追加;超限 tmp+rename;memory_search 只读检索

层	输入	输出	处理(关键步骤)
①快照	MEMORY.md(每 workdir 独立)	安检后的快照文本	Agent 创建时读全文,逐行 _scan() 注入安检,命中 promptware 模式替换占位符
②烘焙	快照文本	system prompt 尾部	BASE_PROMPT + 快照 + skills 索引,创建后冻结 —— 前缀缓存全程命中
③追加	save_lesson(text)	- [UTC ts] text 一行	规范化为单条记录;单次 write 行级原子;相同 workdir 的会话有意共享项目记忆,不同 workdir 不共享
④截断	文件 >8K	保留最近条目	按行保留累计 ≤KEEP_CHARS 的最近条目,tmp+rename 原子替换(读取方不会看到半个文件)
⑤检索	memory_search(keyword)	命中行(≤20)	平文件关键词检索 + 同样安检;容量上限即精度:检索域永远小到关键词足够准

分层描述:快照层负责「会话开始时看到什么」;烘焙层用冻结换缓存;追加层负责「会话学到什么」(下个 Agent 才生效,天然避开 turn 内竞态);截断层防无限增长;检索层补一个只读取数。防线在写入侧与加载侧双重安检,毒条目既不进 system 也不进检索域。

为什么这样设计:记忆快照若随会话内容变化,system prompt 前缀就变了,前缀缓存全部失效 —— 每轮都付全价 token。「冻结 + 尾部注入」用一点信息滞后换取整场会话的缓存命中,是缓存宪法的直接应用。

模块 6 • skills.py 技能(86 行) [L1] • 层:按需

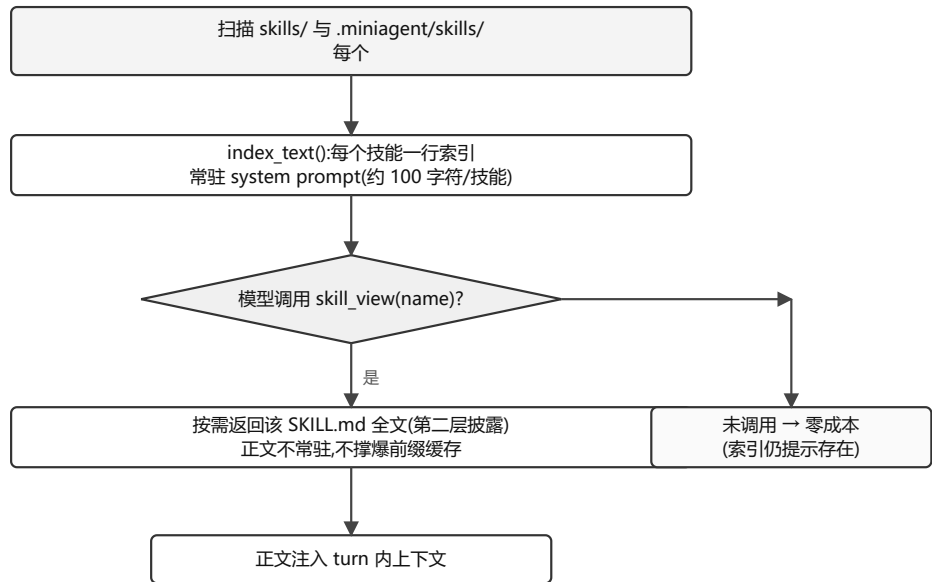


图 6-1 skills.py 知识图谱:扫描 frontmatter -> 索引|一行常驻 system -> skill_view 按需取正文

层	输入	输出	处理(关键步骤)
①扫描	skills/ 与 .miniagent/skills/	技能清单	每个 /SKILL.md 解析 frontmatter(name + description)
②索引	技能清单	index_text() 一行/技能	常驻 system prompt 尾部(约百字符级),提示模型「有这个技能存在」
③取用	skill_view(name)	SKILL.md 全文	正文按需注入 turn 内上下文;未调用则零成本

分层描述:扫描层发现;索引层常驻但极小;取用层延迟绑定。这是 kimi 三层披露的最小两层:索引常驻 + 正文按需,防止提示词膨胀撑爆前缀缓存。

模块 7 • hooks.py 钩子(166 行) [L2] • 层:事件

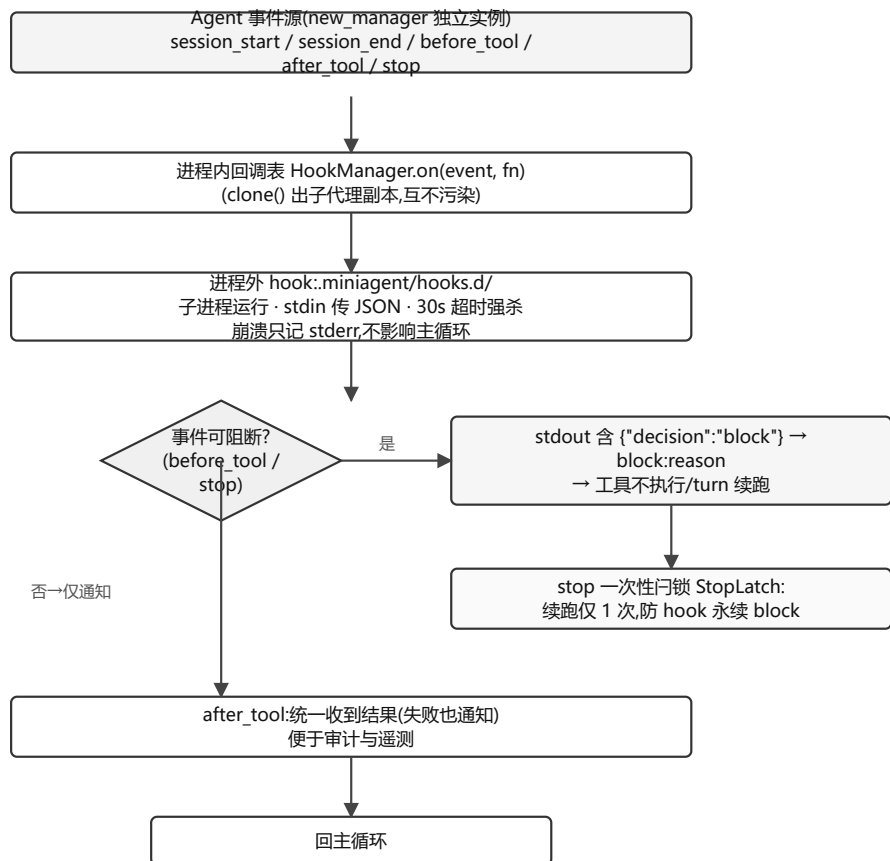


图 7-1 hooks.py 知识图谱:5 事件 → 回调表 + 进程外 hook → 30s 强杀 → 阻断语义 → stop 一次性门锁

层	输入	输出	处理(关键步骤)
①事件源	主循环固定点	5 事件 payload	session_start / session_end / before_tool / after_tool / stop;只有 before_tool 和 stop 可阻断
②实例隔离	Agent 创建	独立 HookManager	new_manager() 从全局配置复制;clone() 出子代理副本;on_event 不污染其他会话
③进程外	hooks.d/_*.py	block:reason / 通过	子进程运行、stdin 传 JSON、30s 超时强杀、崩溃只记 stderr;stdout 打印 {"decision":"block"} 即阻断
④门锁	stop 阻断	续跑 ≤1 次	StopLatch 一次性防「hook 永远 block → turn 永不结束」
⑤可见性	任何工具调用结果	after_tool 统一通知	未知/畸形/权限拒绝不触发 before_tool,但 after_tool 仍收到失败结果 —— 审计不缺角

分层描述:事件层定义契约;隔离层防会话串 hook;进程外层把不可信代码推到子进程并限时;门锁层防死锁;可见性层保证遥测完整。

模块 8 • acp.py ACP 宿主(104 行) [L1] • 层:表面

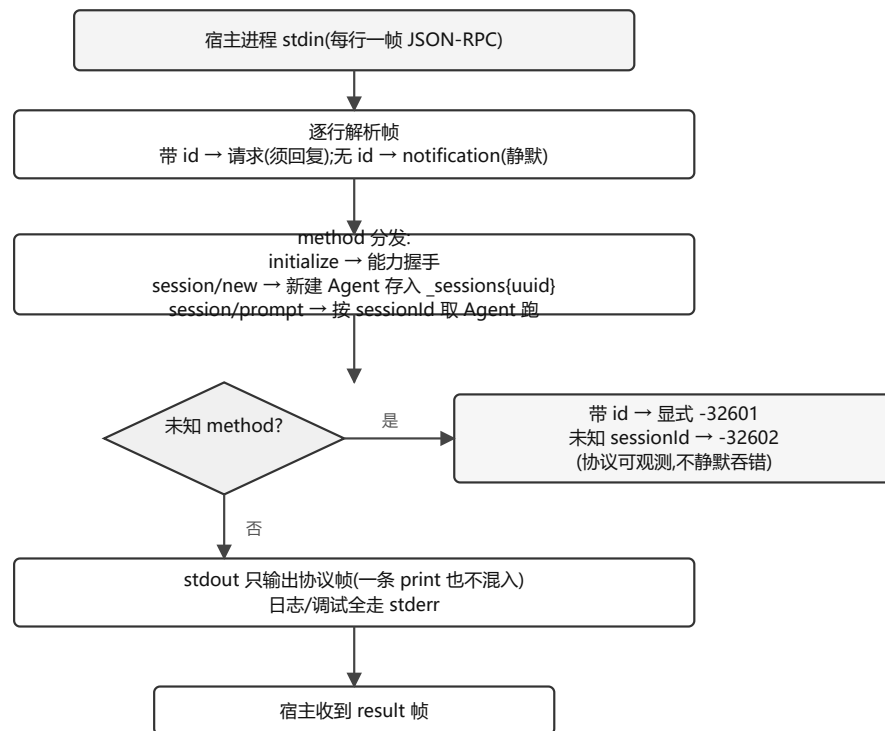


图 8-1 acp.py 知识图谱:stdin 逐行帧 → method 分发 → 多 session 字典 → stdout 纯协议帧/错误显式码

层	输入	输出	处理(关键步骤)
①解析	stdin 每行一帧	请求或 notification	带 id → 请求(须回复);无 id → notification 静默
②分发	method 名	Agent 调用/结果帧	initialize 能力握手;session/new 新建 Agent 存 _sessions{uuid:hex};session/prompt 按 sessionId 取 Agent 跑
③错误	未知 method/sessionId	显式错误码	未知 method → -32601;未知 sessionId → -32602;协议可观测,不静默吞错
④流控	一切输出	stdout 纯协议	stdout 只走协议帧(一条 print 也不混入);日志/调试全走 stderr

分层描述:解析层定帧;分发层维护多 session 字典(每 session 一个独立 Agent,不串线);错误层把失败变成协议内的显式信号;流控层保证宿主能可靠解析 stdout。这是「一条 print 毁掉协议流」事故的反面设计。

模块 9 • session.py 会话持久化(93 行) [L2] • 层:持久

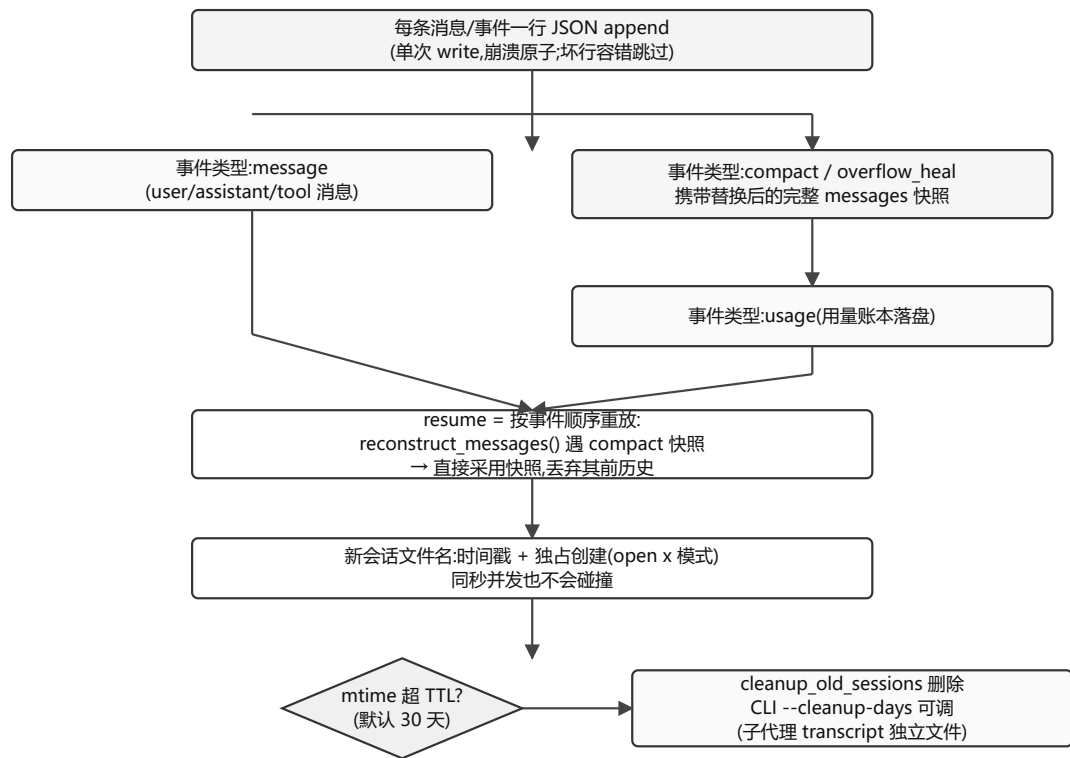


图 9-1 session.py 知识图谱:JSONL 追加(消息/compact 快照/usage)→ 重放遇快照即切换 → 独占创建防碰撞 → TTL 清理

层	输入	输出	处理(关键步骤)
①追加	每条消息/事件	JSONL 一行	单次 write 崩溃原子;坏行容错跳过(resume 时)
②快照事件	compact / overflow_heal	替换性事件	携带替换后的完整 messages 快照 —— 修复了历史「跨进程 resume 丢失压缩摘要」缺陷
③重放	transcript 路径	重建 messages(协议对已修复)	reconstruct_messages() 按事件顺序:message 追加,遇 compact 快照直接采用、丢弃其前历史;末尾_repair_tool_pairs():孤儿 tool 丢弃、缺 result 补占位(防崩溃截断 → 下一条 chat 直接 400)
④命名	新会话	唯一文件名	时间戳 + open(x) 独占创建;同秒并发 Agent 也不会拿到同一路径
⑤清理	mtime TTL	删除计数	cleanup_old_sessions(默认 30 天);CLI --cleanup-days 可调;子代理 transcript 独立文件

分层描述:追加层是「会话即账本」;快照层让压缩成果可持久;重放层让崩溃恢复回到**合法可发送**的真实上下文(协议对完整性修复是 resume 唯一跨进程通路上的最后防线);命名层防并发覆盖;清理层防无限增长。

为什么这样设计:存储选型 JSONL 而非 Redis:只要恢复 → 追加日志够用,要检索才上库。压缩快照一行约放大 3K chars,换取 resume 不重付全量 prompt —— 压测中该缺陷曾致 5 用户多付约 30 次摘要调用,本版已修。

模块 10 • subagent.py 子代理(38 行) [L1] • 层:派生

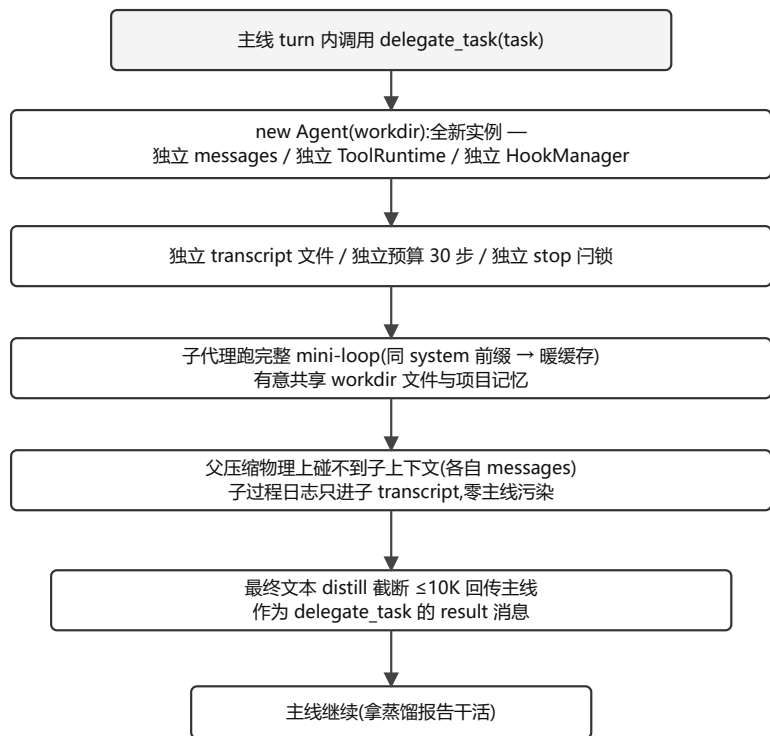


图 10-1 subagent.py 知识图谱:delegate_task → 全新 Agent(六独立)→ 完整 mini-loop → distill ≤10K 回传,过程零主线污染

层	输入	输出	处理(关键步骤)
①派生	delegate_task(task)	全新 Agent 实例	独立 messages / ToolRuntime / HookManager / transcript / 预算 30 步 / stop 门控
②执行	子任务描述	完整 mini-loop	同 system 前缀(暖缓存);有意共享 workdir 文件与项目记忆 —— 能操作同一项目,隔离的是上下文与运行时绑定
③回传	子代理最终文本	distill ≤10K	截断回传作为 result 消息;探索过程只进子 transcript,主线零污染

分层描述:派生层物理隔离上下文(父压缩碰不到子);执行层跑完整循环;回传层只给结论不给过程。对照 Hermes #38727 事故(父与 fork 共享 session 同时压缩 → 历史分叉):本设计从结构上免疫。

模块 11 • llm.py LLM 客户端(114 行) [L2] • 层:通信

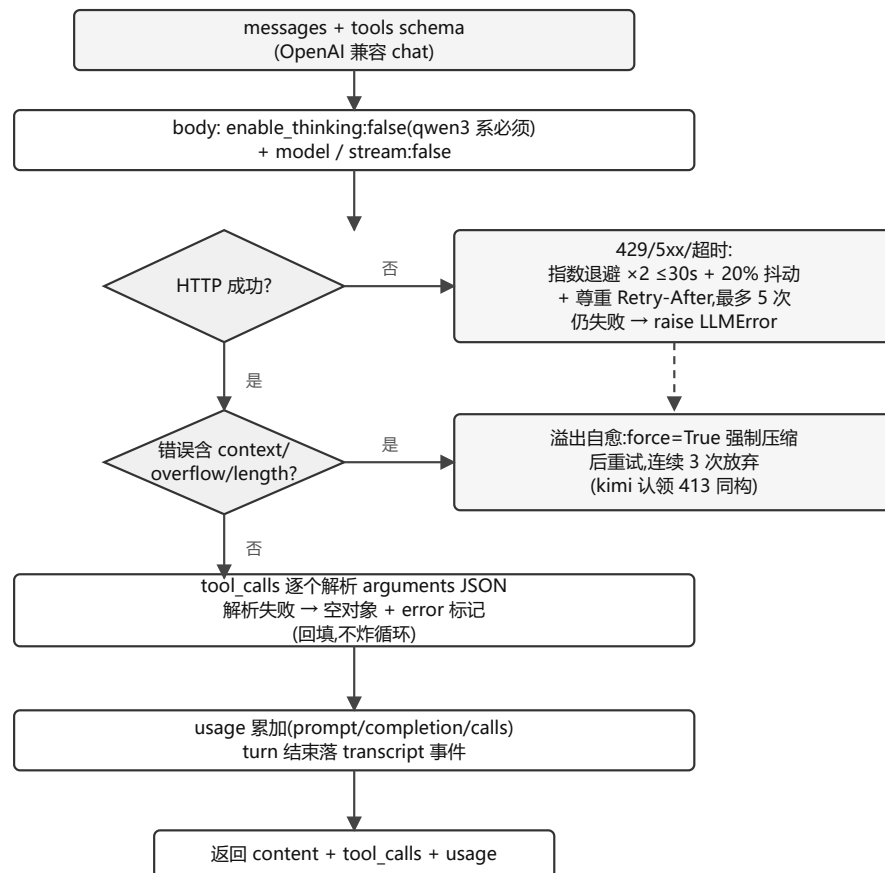


图 11-1 llm.py 知识图谱:enable_thinking:false → 429/5xx 退避 ≤5 → 溢出强制压缩重试 ≤3 → arguments 解析降级 → usage 账本

层	输入	输出	处理(关键步骤)
①请求	messages + tools schema	HTTP 请求	OpenAI 兼容 chat;enable_thinking:false 放 body 顶层(qwen3 系必须,否则混入思考 token)
②重试	429/5xx/超时	退避后重发	指数退避 ×2 上限 30s + 20% 抖动 + 尊重 Retry-After,最多 5 次;仍失败 raise LLMError
③溢出自愈	错误含 context/overflow/length	压缩后重试	force=True 强制压缩再试,连续 3 次放弃(kimi 认领 413 同构;loop_chat 驱动)
④解析	resp.content/tool_calls	结构化结果	arguments JSON 解析失败 → 空对象 + error 标记,作为工具错误回填,不触发 hook/handler
⑤账本	resp.usage	累计计数	prompt/completion/calls 每步累加,turn 结束落 transcript(成本可审计)

分层描述:请求层管协议兼容;重试层管瞬时故障;自愈层管窗口溢出;解析层把模型畸形输出降级为普通错误;账本层把成本变成数据。五层都不改变「失败也是一种结果」的自愈语义。

模块 12 • cli.py 终端表面(106 行) [L1] • 层:表面

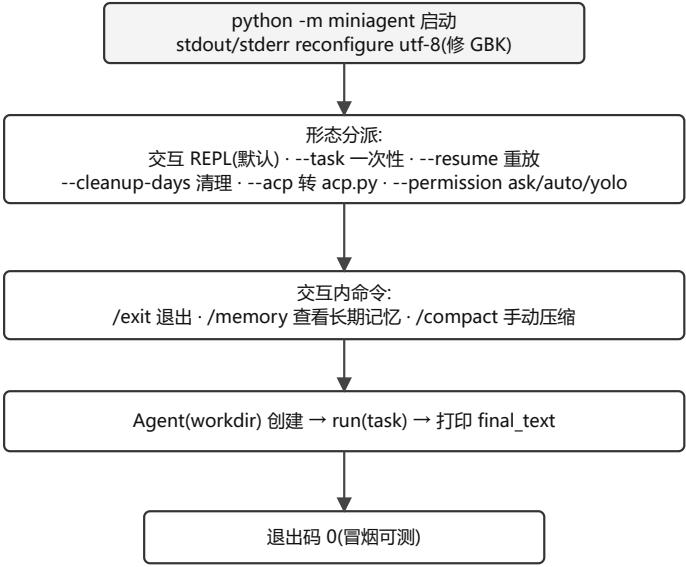


图 12-1 cli.py 知识图谱:启动 reconfigure utf-8 → 形态分派 → 交互命令 → Agent.run → 退出码

层	输入	输出	处理(关键步骤)
①启动	python -m miniagent	utf-8 输出流	stdout/stderr reconfigure utf-8(修复 Windows GBK 输出 emoji 崩溃)
②分派	argv	运行形态	交互 REPL(默认) · --task 一次性 · --resume 重放 · --cleanup-days 清理 · --acp 转 acp.py · --permission ask/auto/yolo
③交互命令	REPL 输入	本地动作	/exit 退出 · /memory 查看长期记忆 · /compact 手动压缩(走 Agent.compact 落快照)
④执行	任务文本	final_text	Agent(workdir).run(task) → 打印;退出码可冒烟

分层描述:启动层修编码;分派层把五种形态收敛到一个入口;命令层给交互用户三个逃生口;执行层只是 Agent 的薄壳 —— 表面层不含任何策略。

模块 13 • web.py Web 表面(171 行) [L1] • 层:表面

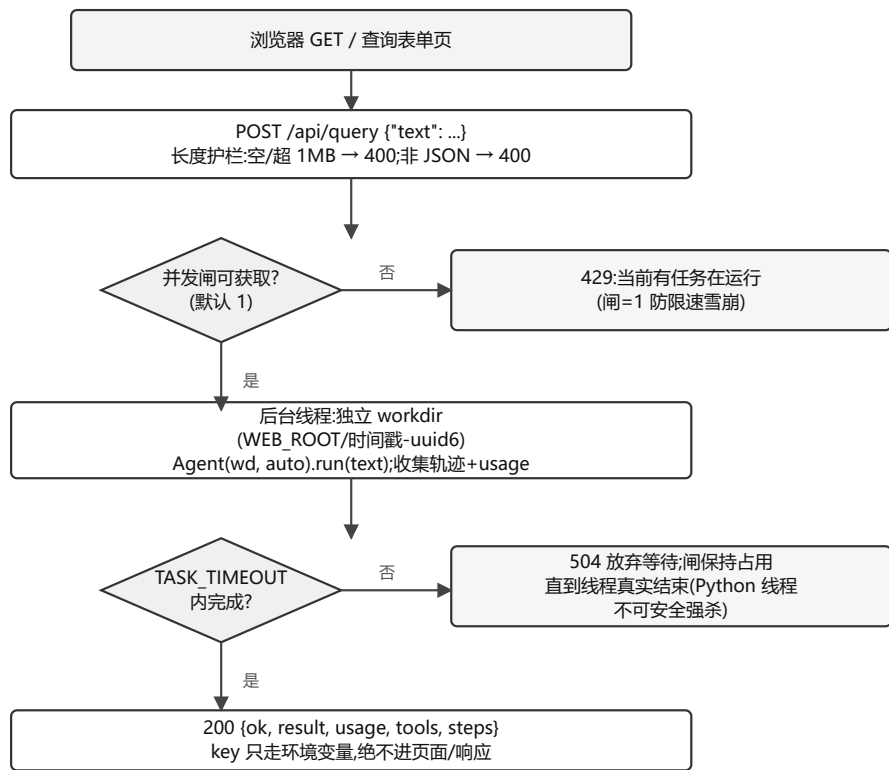


图 13-1 web.py 知识图谱:表单页 → POST /api/query → 并发闸(1) → 独立 workdir 后台线程 → 超时放弃等待但闸保持占用 → 200/429/504

层	输入	输出	处理(关键步骤)
①表单	GET /	HTML 表单页	纯静态表单;key 只走环境变量,绝不进页面/响应
②护栏	POST /api/query	400 或放行	请求体长度护栏(空/超 1MB 拒;非法 JSON 拒)
③并发闸	信号量(默认 1)	429 或进入	当前有任务运行 → 429 请稍后(防 API 限速雪崩)
④执行	任务文本	result/usage/tools	后台线程:独立 workdir(WEB_ROOT/时间戳-uuid6);Agent(wd, auto).run;收集工具轨迹与用量
⑤超时	TASK_TIMEOUT(600s)	504	放弃等待;Python 线程不可安全强杀 → 闸保持占用直到任务真实结束,避免超时任务与新任务重叠

分层描述:表单层零状态;护栏层拒畸形请求;闸层控并发;执行层隔离 workdir 防请求串目录;超时层诚实处理「杀不掉的线程」—— 这是并发闸超时失效问题的正面设计。

2.4 跨模块交互:token 漂移 • 多源会话 • context × memory

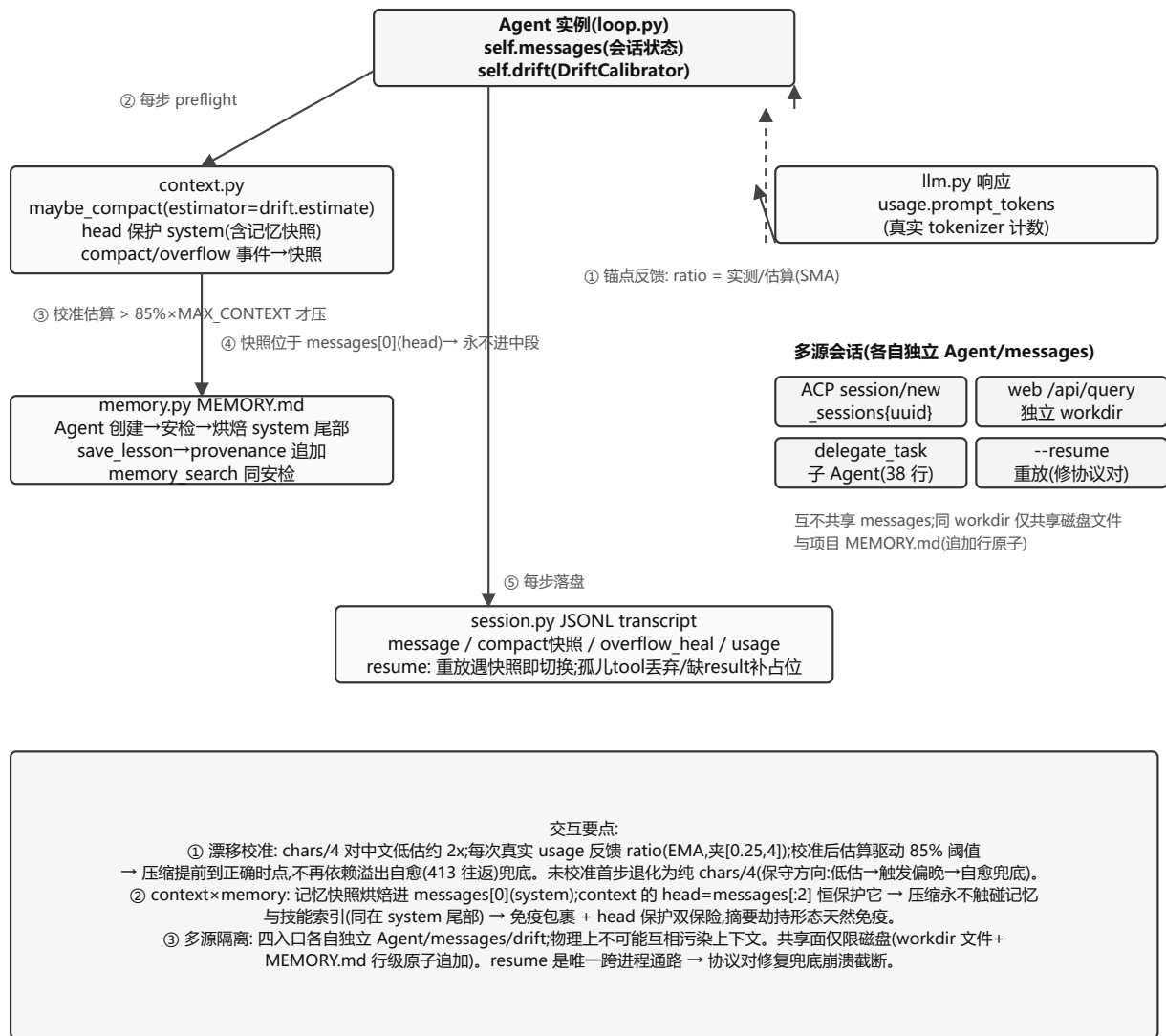


图 2-2 跨模块交互视图:校准回路(loop→llm usage→context 阈值)· head 保护(memory 快照永不进中段)· 四入口多源会话(各自独立 Agent/messages,共享面仅限磁盘)

token 漂移(本版新增)。chars/4 对英文近似成立,对中文低估约 2x(Qwen tokenizer 中文 ≈1.5~2 字符/token)。未校准的后果链:估算低估 → 85% 阈值触发偏晚 → 溢出自愈兜底(一次 413 往返 + 全量重压)。本版在 context.py 加 **DriftCalibrator**:每次 _chat 后用真实 usage.prompt_tokens 对刚送出的 messages 做锚点校准(ratio = 实测/估算,EMA α=0.3,单点夹 [0.25,4] 防畸形),校准系数直接驱动 maybe_compact 的阈值判定与 tail 预算。单测覆盖:首锚点设值/EMA 收敛/夹逼/非法锚点忽略/校准估算。**为什么不用阻塞式测量**:kimi 的双层计量要在关键点插 token 计量 job;本设计用顺路必然返回的 usage 做零成本校准,逐步未校准时退化为纯 chars/4(偏差方向:低估 → 触发偏晚 → 自愈兜底,不会误压),第二步起即有校准。

多源会话(四入口)。ACP session/new_sessions(uuid→Agent)、web /api/query(每请求独立 workdir→独立 Agent)、delegate_task(全新子 Agent)、--resume(重放重建)。四入口**各自持有独立 messages / ToolRuntime / HookManager / DriftCalibrator**,物理上不可能互相污染上下文;共享面仅限磁盘 —— 同 workdir 的项目文件与 MEMORY.md(行级原子追加 + tmp+rename 截断,多进程安全)。resume 是唯一跨进程上下文通路,因此其重放路径加协议对完整性修复:**孤儿 tool 消息丢弃、残缺 assistant(tool_calls) 补占位 result** —— 崩溃截断的 transcript 若原样重放,下一条 chat 直接 400 invalid tool message。单测覆盖孤儿丢弃/完整对零改动。

context × memory 交互。记忆快照在 Agent 创建时经注入安检后烘焙进 messages[0](system)尾部并冻结;context 的 head = messages[:2] 恒含它 → **压缩物理上永不触碰记忆与技能索引**;中段摘要再裹 [REFERENCE ONLY] 免疫包裹 → 「摘要劫持」形态双保险免疫。反向交互:save_lesson 写入只发生在会话中(下个 Agent 生效),不触发任何 system 变更 → 前缀缓存全程命中。memory_search 是唯一读取侧通路,同样过 _scan 安检 —— 毒条目既不进 system 也不进检索域。

3、如何运行

本机 Python / Docker / ACP 宿主 / Web 四种形态;API key 只走环境变量,绝不进代码与镜像。

3.1 环境变量

变量	说明	默认
MINIAGENT_API_KEY	LLM API key(必填,运行时注入)	—
MINIAGENT_BASE_URL	OpenAI 兼容接口	DashScope compatible-

		mode
MINIAGENT_MODEL	模型名	qwen3.7-max
MINIAGENT_WORKDIR	沙盒围栏根目录	当前目录
MINIAGENT_PERMISSION	ask / auto / yolo	auto
MINIAGENT_MAX_CONTEXT	上下文窗口 tokens(阈值 85%)	300000
MINIAGENT_MAX_STEPS	每 turn 步数预算	60
MINIAGENT_SUBAGENT_MAX_STEPS	子代理独立步数预算	30
MINIAGENT_MAX_INPUT_CHARS	入口最大输入字符(超限拒)	200000
MINIAGENT_WEB_PORT/_TIMEOUT/_CONCURRENCY/_ROOT	Web 表面层参数	19120 / 600 / 1 / 临时目录

3.2 四种形态

```
# 交互模式(/exit 退出,/memory 看记忆,/compact 手动压缩)
python -m miniagent --workdir /path/to/project

# 一次性任务
python -m miniagent --workdir ./proj --permission auto --task "创建 utils.py 并写好 unittest"

# 恢复会话(重放 transcript 继续,含压缩快照)
python -m miniagent --workdir ./proj --resume .miniagent/sessions/<ts>.jsonl

# Docker
docker run --rm -v /tmp/ws:/workspace -e MINIAGENT_API_KEY=$KEY miniagent:latest \
  --task "写一个记账 CLI 并跑通 unittest"

# ACP 宿主(stdout 只走协议帧,日志全在 stderr)
python -m miniagent --acp

# Web 表面层(HTTP 查询表单,并发闸+超时+每请求独立 workdir)
python -m miniagent.web

# 测试(65 个离线测试,FakeLLM/Mock 不触网)
python tests/test_all.py
```

3.3 扩展点

扩展	位置	说明
自定义技能	skills/<name>/SKILL.md	frontmatter 写 name/description,正文即操作手册
进程外 hook	.miniagent/hooks.d/<event>_*.py	stdin 收 JSON;stdout 打印 {"decision":"block"} 即阻断
新工具	tools.py	@tool 装饰器在 Agent 创建前注册;schema/只读属性自动进入 ToolRuntime 与 sandbox

工具注册示例:from miniagent.tools import tool → @tool("add_numbers", "计算两个整数之和。", {schema}, readonly=True) → 注册必须在创建 Agent 前完成;调用按「解析错误/注册表/参数校验 → sandbox → before_tool → handler → after_tool → 结果治理」执行。

4、对比报告:miniagent vs kimi-code vs dsh vs Hermes

教程对照的三个工业 codebase:kimi-code v0.34、DeepSeek Harness(dsh v0.1.0-rc.5)、hermes-agent 0.18.0。四方设计理念一句话:miniagent=最小完备集; kimi-code=有人值守 CLI;dsh=everything is a plugin;Hermes=无人值守平台。

4.1 全模块对照表

模块	miniagent	kimi-code	dsh (DeepSeek)	Hermes
主循环	while 三行语义+预算硬墙	turn/step 队列+错误处理器插件链	全仓唯一具体循环(loop-agent);取消与静止是一等公民	预算驱动 while+refund 记账
上下文	单层估算 85%+头尾保留	双层计量(锚点实测+估算)+阻塞式 job	三层装配线;上下文预算守门人独立成模块	四阶段压缩+三重反抖动+压缩锁
工具	字典注册表+结果治理	三层折叠+双 policy 正交	插件即工具;268 子包按 50 功能域组装	AST 自动发现+足迹 6 级阶梯
沙盒	hardline+去混淆+三分级(进程内)	12 节静态权限链(进程内,微秒级)	进程级文件效果沙盒+E2B 远程执行世界	8 层纵深+Smart Approval+网络出口隔离
记忆	MEMORY.md 冻结快照+安检	无跨会话记忆(有意)	无 memory 模块(有意):日志即记忆	双路注入+fork 巩固+漂移检测

技能	frontmatter 两层披露	双路激活+wire 记账	三件套+Provider 契约+双层注册表	纯文档契约+紧/松双闭环 curator
子代理	delegate_task 单点派生	扁平子代理+swarm 限速	编排层独立成章:6 provider+Workflow+ Ralph	fork 5 重隔离
持久化	JSONL 追加+快照重放	wire.jsonl 事件溯源	SessionEvent 日志,"logged" 不变量	SQLite 双 FTS5+软删归档
设计理念	最小完备,痛了再加	确定性优先,用户在场	归约主义:拒绝新状态容器	纵深防御,无人值守

4.2 三点结构性观察

① 记忆的四种答案是四方分歧最大的地方 —— miniagent 平文件快照 / kimi 干脆不要 / dsh 用「没有 memory 模块」这个否定本身作答案(日志即记忆)/ Hermes 上双路注入+巩固流水线,选哪条路线取决于「无人值守程度」与「跨会话学习收益」的权衡。② 沙盒的进程边界是第二个分水岭:miniagent/kimi 在进程内做规则裁决(快但同进程),dsh/Hermes 把执行推到进程级沙盒(慢一档但爆炸半径小一档)。③ 插件化的代价在 dsh 上可见:268 子包换来可整包替换的 loop,认知负担巨大 —— miniagent 的单包是刻意的反面选择。

4.3 行为差异与选型结论

维度	miniagent	kimi-code	Hermes
代码量	核心约 1,500 行(15 文件)	数万行(loopService 1,222 行)	更大(conversation_loop 5,294 行)
延迟	非流式整段返回	流式优先,首 token 一等指标	事件回调外发
失控兜底	max_steps 硬墙+stop 门锁	插件链	预算硬墙+refund+退出诊断
无人值守	有限(bash 需外套容器)	低(设计前提:用户在终端前)	高(为无人值守设计)
跨会话学习	save_lesson+冻结快照	零(外包给用户)	全自动(fork+curator)
可审计	JSONL 全量可查	wire.jsonl 事件溯源	SQLite+FTS5 全文检索

什么时候选谁:miniagent 教学与二次开发基座,每个模块一眼看完,能力缺口(流式、swarm 调度、反抖动、MCP)是有意留白。kimi-code 型有人盯着的编程 CLI,延迟与确定性优先。Hermes 型无人值守多表面平台,安全边界与状态可恢复优先,代价是数倍代码量。

5、问题与解决

两类:结构性问题(对照教程原型事故,证明设计免疫或已修复)+ 实测问题(时间序修复清单)。

5.1 问题一:多子代理上下文压缩污染 —— 不存在,且从设计上免疫

教程原型事故是 Hermes #38727:父 agent 与后台 fork 共享同一 session,两边同时触发压缩 → 历史分叉。miniagent 的 delegate_task 派生全新 Agent 实例(独立 messages / 独立 ToolRuntime / 独立 HookManager / 独立 transcript / 独立 30 步预算 / 独立 stop 门锁),父压缩物理上碰不到子上下文:只回传 distill 报告(10K 截断);system prompt 同前缀缓缓存。现场验证 T7:父 transcript 只有 delegate_task→write_file→bash×2,子代理 11 次探索调用记录在自己独立 transcript 里,主线零污染。

5.2 问题二:多会话记忆污染 —— 一种天然免疫,两种已修复

形态 A(摘要劫持):天然免疫 —— 记忆快照在 messages[0](system),head 保护永不进中段;摘要带 [REFERENCE ONLY] 免疫包裹。形态 B(并发写截断竞态):已修 —— 追加写带 UTC provenance(行级原子);截断改 tmp+rename 原子替换。形态 C(毒条目带毒注入):已修 —— 快照加载与 memory_search 逐行注入安检,命中条目替换占位符;单测确认毒条目被过滤、正常条目不受影响。

为什么这样设计:为什么不学 kimi 不要记忆:kimi 把跨会话学习整体外包 —— 零污染面但也零自学习。miniagent 保留轻量自产记忆,代价是支付 B/C 两项防御。

5.3 实测问题与修复清单(时间序)

#	问题	根因	修复	验证
1	一次性任务结束打印崩溃	Windows GBK,输出含 emoji	cli.py 启动 reconfigure utf-8	冒烟 exit 0
2	进程外 hook 读 payload 失败	子进程 stdin 默认 GBK	buffer+utf-8;注入 PYTHONIOENCODING	T4b 实测
3	单 user 会话永不压缩	守护把 cut 拉回 1→middle 恒空	守护仅 last_user_idx≥2 生效+回归测试	t6d 触发 2 次
4	记忆并发写竞态/带毒注入	见形态 B/C	provenance+tmp+rename+安检	TestMemory ×3
5	无子代理机制(能力缺口)	最小设计留白	delegate_task 物理隔离派生	TestSubagent+T7
6	跨进程 resume 丢失压缩摘要(压测问题 A)	compact 事件不落摘要文本;每轮独立进程全量重灌	compact/overflow 事件携带完整 messages 快照;重放遇快照即切换	57/57 回归含快照重放用例
7	会话工具/hook 全局串绑	模块级单例注册表	ToolRuntime/HookManager 按 Agent 实例化	隔离回归测试
8	并发会话文件名同秒碰撞	时间戳命名	open(x) 独占创建原子占位	并发回归
9	web 超时后并发闸失效	超时即释放闸 → 超时任务与新任务重叠	闸保持占用到线程真实结束	web 回归

10	sandbox 只读名单与注册表漂移	两份名单各自维护	known_tools 由 ToolRuntime 生成(单一事实源)	注册回归
11	token 漂移:chars/4 中文低估 ~2x,压缩触发偏晚靠 413 自愈兜底	估算无真实反馈	DriftCalibrator:每次真实 usage 锚点 EMA 校准,驱动 85% 阈值与 tail 预算	TestDriftCalibrator ×5 + 接线测试
12	resume 重放孤儿 tool 消息 → 下一条 chat 400	崩溃截断致协议对残缺	_repair_tool_pairs():孤儿丢弃/缺 result 补占位	TestResumeProtocolRepair ×2

5.4 已知边界(有意不修,对应演化阶梯)

极小窗口压缩 thrash(第 6 级反抖动三件套) · LLM 非流式(第 1 级) · 无并发子代理调度 swarm(第 5 级) · 无 MCP 外部工具桥接(第 8 级) · 无 LLM 兜底裁决 Smart Approval(第 10 级) · bash 不是 OS 级沙盒(无人值守应套容器/E2B) · 同一 workdir 父/子代理有意共享项目文件与 MEMORY.md(隔离的是上下文与运行时绑定)。

6、工程问答(16 题)

题目来自 DeepSeek Agent 岗三面连环追问。标注：〔已有机制〕现场验证过 · 〔针对性补全〕本轮新增 · 〔参照〕kimi/Hermes 对照。

6.1 一面 · 工程细节题

Q1 上下文用 message window 还是 token window?〔已有机制〕**token window**。context.py 以估算 token 数(chars/4)对 30 万窗口设 85% 触发线,message window 无法表达「10 条消息里有一条 240KB 工具输出」的真实分布(实测单条 read_file 结果 25 万字符)。代价是估算漂移,工业解法是 kimi 双层计量;miniagent 停在单层粗估+85% 保守阈值+溢出自愈兜底。{kimi: 双层计量 · Hermes: 三套信号协调}

Q2 会话持久化过期策略?并发会话冲突?〔针对性补全〕存储选型 **JSONL 而非 Redis**(只要恢复+追加日志够用,要检索才上库)。过期:cleanup_old_sessions(默认 30 天)按 mtime TTL。并发冲突:多会话同 workdir 写 MEMORY.md → 行级原子追加+tmp+rename;多子代理共享上下文压缩 → 物理隔离(独立 transcript);同秒新建会话 → 独占创建防碰撞。{kimi: AppendLogStore cutover · Hermes: SQLite 压缩锁 TTL 300s}

Q3 长期记忆用向量库吗?检索越来越不准怎么办?〔针对性补全〕**不用向量库**—— 记忆是 8K 字符硬上限的平文件(容量即精度:上限强制淘汰旧条目,检索域永远小到关键词足够准)。memory_search 只读检索。防线在写入与加载双侧:provenance 溯源+注入安检,毒条目不进检索域。记忆条目破千、跨项目复用时才上向量库。{Hermes: FTS5+trigram}

Q4 上下文溢出兜底?〔针对性补全〕三层:① 预防 —— preflight 超 85% 即压缩;② 自愈 —— provider 报 context/overflow/length 错误时 _chat() 捕获,force=True 强制压缩后重试,连续 3 次放弃(且溢出事件本身落快照);③ 兜底 —— 摘要失败 fail-open 保原会话。{kimi: 认领 413,窗口 ×0.85}

Q5 单用户日均 token 成本?百万级怎么控制?〔针对性补全〕用量账本(prompt/completion/calls 累加,turn 结束落 transcript)。现场实测:轻任务 = 5,838 prompt + 189 completion / 4 次调用;完整项目任务约 14 次工具调用、~10 次 LLM 调用。三板斧:① 缓存宪法(前缀只追加);② 结果治理(50K 落盘只回 2K);③ 预算硬墙。{kimi: 分段缓存 · Hermes: 4 断点+冻结快照}

Q6 恶意刷长文本怎么限制?〔针对性补全〕三层:入口 MAX_INPUT_CHARS(默认 20 万)超限直接拒,零 **LLM 消耗**;工具侧 bash 120s/30K 截断/50K 落盘;循环侧 max_steps 硬墙+stop 门锁。Web 形态另有请求体长度护栏。

Q7 为什么基座不微调?拿什么数据判断?〔设计决策〕行为约束都在工程层(system prompt 纪律+工具协议+权限链),不需要权重级定制。判断数据:工具调用成功率(73 次 0 协议错误)、任务完成率、失败自愈率、单任务成本(账本)。微调唯一值回票价的是把 BASE_PROMPT 烧进权重省 token —— 但损失迭代速度,不划算。

Q8 讲一个线上故障排查。〔真实案例〕**单 user 会话压缩失效**:小窗口压测(MAX_CONTEXT=1500)下压缩恒 0。逐步重放 transcript 计算各时点估算,确认第 3 条消息起已超阈值却不触发 → 定位到「最后一条 user 必须在 tail」守护把 cut 强制拉回 1,middle 恒空。修复:守护仅 last_user_idx≥2 生效。效果:压缩触发 0→2 次;回归测试防复发。教训:守护规则必须考虑「被保护对象已在保护区」的边界。

6.2 二面 · 系统设计题

Q9 模块怎么拆分?通信?〔已有机制〕13 个模块按「窄腰核心+边上插拔」拆:loop 是唯一有状态核心,三个窄接口通信 —— 工具协议(注册表快照)、事件表(hooks 5 事件)、消息列表(会话状态,正常只追加)。不走消息总线是为了缓存宪法:总线式动态注入会打碎前缀缓存。

Q10 简单任务 vs 多步拆解怎么判断?〔已有机制〕拆解权交给模型,工程层只设边界 —— max_steps 硬墙(60)、子代理独立预算(30)、delegate_task 让模型把探索性子任务显式派生。两种工业路线(kimi 拆解权给模型 / Hermes 预算 refund)都验证了「不要在工程层替模型做拆解判断」。

Q11 执行一半失败:重试/回滚/重新规划?〔已有机制〕分四层,触发条件显式:

失败类型	策略	触发条件
工具执行失败	错误文本回填,模型自行重试或换路(T1 实测)	任何 tool exception
429/5xx/超时	指数退避(×2 上限 30s+20% 抖动),最多 5 次	HTTP 状态码/连接异常
上下文溢出	强制压缩后重试,连续 3 次放弃(落溢出快照)	错误含 context/overflow/length
权限拒绝/hook 阻断	不重试不绕过 ,原因回填换方案(T4b 实测)	sandbox/hook veto

回滚在最小设计里刻意没有(文件操作无事务),对应 kimi 的 undo/wire Op 重放 —— 痛了再加。

Q12 多工具联动?权限管控?防越权?〔已有机制〕联动:纯注册表快照,顺序由模型按 schema 编排;结果治理保证单工具不灌爆窗口。权限五道防线:① hardline 硬线(yolo 也不放行);② 去混淆防绕过;③ 未知工具 fail-closed;④ 三分级+ask/auto/yolo;⑤ workdir 围栏+环境剥离。价值观:**危险检测先于用户授权**。{Hermes: hardline 先于 yolo bypass}

6.3 三面 · 认知题

Q13 Agent 长期风口还是短期热点? 长期。模型每代跃迁,但三个工业 codebase 的厚度证明**工程表现的上下限由 Harness 决定**——同一模型接不同 Harness,安全性/成本/长任务存活率差一个数量级。

Q14 核心竞争力? 能从「事故」反推架构:每个模块都能回答「防什么事故、对应演化阶梯第几级、kimi/Hermes 怎么解」。demo 开发者堆框架,工程师管爆炸半径、管成本、管最坏情况下限。

Q15 1-2 年最大瓶颈?ToB 还是 ToC? 工程能力。模型已「够用」,卡脖子的是长会话成本、无人值守安全、状态可恢复。ToB 先跑通——容错预算高、边界清晰、权限可预声明;ToC 越权风险面不可控。

Q16 从 0-1 带 Agent 业务线,第一步? 先定形态再写代码:有人值守还是无人值守?这决定主循环形状、安全厚度、存储选型。招人按模块招,评审标准就是「每份复杂性答得出防什么事故」。

7、压力测试与问题发现(2026-08-16 实测 + 修复对照)

三组实测:A 组 3 个编译类任务、B 组 5 个复杂编程任务、C 组 5 用户×20 轮并发多轮对话(强制小窗口压 context/memory 压缩)。全部现场真跑,发现 6 项问题;其中问题 A(resume 丢失压缩摘要)已在 690eabf 修复,其余按优先级保留。

7.1 实测任务与结果总览

#	任务	结果	说明
C1	C 词频统计程序(检测 gcc/tcc/cl 并编译)	降级完成	本机无 C 编译器→诚实报告,Python 等价实现验证 the=3
C2	LaTeX 中文短文 report.tex,xelatex 编译	通过	两轮编译(第二轮解决交叉引用),PDF 1 页 41KB,0 error
C3	Python 包 mylib(setup.py)+pip editable 安装+断言	通过	fib(10)=55 / is_prime(97) / sort 全过
P1	JSON 解析器(纯 Python,25 测试)	通过	独立复跑 29 tests OK(含 \uXXXX 代理对/前导零拒绝)
P2	Markdown→HTML 渲染器(20 测试)	通过	标题/嵌套列表/围栏代码块/行内格式全覆盖
P3	线程安全 LRU+TTL 缓存装饰器(15 测试)	通过	10 线程×100 次并发读写正确
P4	表达式求值器(递归下降,18 测试)	通过	变量赋值/一元负号/比较/除零与未知变量错误
P5	优先级队列+SQLite 持久化(15 测试)	通过	崩溃恢复顺序不变;重试 3 次入死信;backoff 生效
X1	5 用户×20 轮并发,MAX_CONTEXT=12000 强制压缩	100/100 轮 exit 0	总墙钟 99s;暴露 4 项真实问题(下)

7.2 问题 A(已修复):跨进程 resume 丢失压缩摘要

现象:压测 transcript 显示 u1 的 6 次 compact 事件全部集中在第 17-20 轮,且每次压缩时点估算只有 ~2,950 tokens——远低于 85% 阈值 10,200。

根因:compact 事件只落 {"type":"compact","tokens":...},摘要文本不落盘;压测每轮独立进程(--resume 续),每个新进程都把全量历史重新灌入活上下文——前一轮压缩成果作废,要么重付全量 prompt,要么重新调一次摘要。**代价:**5 用户合计多付约 30 次多余摘要调用。**修复(690eabf):**compact/overflow_heal 事件携带压缩后完整 messages 快照;reconstruct_messages() 遇快照直接采用、跳过其前历史。resume 回到压缩后的真实上下文。

7.3 问题 B(随 A 缓解):长会话成本爬升未压平

20 轮 prompt 合计 99,880 tokens(轮均 4,994,峰值 11,497 ≈ 整窗);5 用户×100 轮合计 501,841 tokens。根因同问题 A(resume 全量重放)。修 A 之后预期轮均降至 ~3K 并走平;若仍不达标,演化级是 kimi 的「锚点实测+估算」双层计量(把 20% 保守余量收窄)。

7.4 问题 C(设计权衡):压缩摘要粒度粗,早轮细节丢失

第 20 轮回顾题的答案只保留主题级信息,第 1-4 轮的具体返回码值(0x0011/0x0022/...)丢失。单次无结构摘要长 middle 一次压完必然有损。可选改进:摘要提示词按轮次/主题分段保留关键数值;kimi 路线是锚点+分段摘要。注意这是设计权衡而非纯缺陷——记忆系统本就是为跨压缩保留关键事实而设,压测中 5 用户各 4 条 save_lesson 全部带 UTC provenance 正确落盘。

7.5 问题 D(已知边界):小窗口下 turn 内压缩抖动

MAX_CONTEXT=12000 时,第 5/10 轮的多步工具调用在阈值边缘反复触发:prompt 序列 6,985→3,957(压)→4,385→4,813→...→11,497→3,328(又压)——单 turn 内两次压缩、两次摘要调用。这是教程 Ch.2 第 6 级「反抖动三件套」(-1 哨兵/无效计数/600s 冷却)的缺席,属已知边界,本次实测量化了代价:第 10 轮多花约 8K tokens。生产窗口 30 万下抖动概率低,优先级低于问题 A。

7.6 问题 E(边界,行为正确):无 C 编译器时诚实降级

C1 任务本机无 gcc/tcc/cl,agent 明确声明「未执行任何编译操作」,交付可编译源码+Python 等价实现完成功能验证——没有假装编译。这是反假阳价值观的正面案例,记录为部署边界:需要 C 工具链的任务请自带环境。

7.7 问题 F(口径问题):测试文件位置约定

P4 把测试写在 tests/ 子目录,根目录 python -m unittest discover 报 NO TESTS,需 discover -s tests 才能跑(测试本身 18/18 全过)。agent 自报口径与独立验证口径不一致。改进:BASE_PROMPT 约定测试文件写在根目录 test_*.py,或任务描述显式指明。

7.8 压测方法(可复现)

```
# _stress/concurrency_test.py: 5 进程 = 5 用户,独立 workdir/transcript
# 每轮注入 ~2.7K 字符背景材料,MAX_CONTEXT=12000 强制压缩
# 每 5 轮一个回顾+save_lesson 任务轮;每轮 --resume 续会话
```

```
# 统计:每轮 exit/耗时/usage + compact 事件 + MEMORY.md 终态
# 结果:conc/summary.json + u*.log;墙钟 99s,100/100 exit 0
```

— 报告完 — 生成:build_miniagent_report.py(XeLaTeX 版已由本 HTML+Chrome 管线取代);素材:《Agent 模块化设计教程 v2》· kimi-code v0.34 · hermes-agent 0.18.0 · miniagent commit 690eabf